

Pipeline Builder

Samuele Mazziotti

2026

Contents

1	Introduction	2
2	Overview and General Functioning of the System	3
3	Software Architecture of the System	4
3.1	API (access and orchestration layer)	4
3.1.1	Central router	4
3.1.2	API Structure	4
3.2	Algorithm Catalog	5
3.3	Persistence Layer (DB)	7
3.3.1	Database Configuration	7
3.3.2	ORM Models	7
3.3.3	Operational Modules	8
3.4	Runtime Models	9
3.5	Validation Schemas	11
3.6	User Interface	12
3.6.1	General Structure of the Interface	13
3.6.2	Context Management	13
3.6.3	ObjectType and Algorithm Management	14
3.6.4	Pipeline Builder	15
3.6.5	Management of Saved Pipelines	16
3.7	System Entry Point	16
4	Usage Examples	17
4.1	Scenario 1: Generic Pipeline	19
4.2	Scenario 2: Performance Analysis	22
4.3	Scenario 3: Reuse of Saved Pipelines	24
5	Conclusions	25

1 Introduction

In recent years, the increasing adoption of Data Science and Machine Learning techniques has highlighted the critical role of data quality in the success of analytical processes. Indeed, noisy, incomplete, inconsistent, or biased data can undermine the reliability of the results [1]. For this reason, data preparation is a crucial phase, as it transforms raw data into usable information. However, this activity is often complex and difficult to automate. Moreover, many existing tools focus on individual operations, such as data cleaning or transformation, without fully supporting the construction of structured and reusable pipelines [1, 2].

The literature also emphasizes the importance of data semantics, namely the ability to explicitly represent the meaning of data, in order to improve the integration, interpretability, and quality of preparation processes [2]. A further reference is the work by Ciammaichella and Ciuffreda, which proposes a computational pipeline for the analysis of multimedia data related to the KPIs of the New European Bauhaus. This work highlights the importance of organizing pipelines into sequential, modular, and interpretable phases, including preprocessing, feature extraction, semantic analysis, and context awareness [3].

In light of these considerations, there is a need for tools that support the composition of end-to-end pipelines in a clear, modular, and guided manner.

This project addresses this need by designing and developing a software system called **Pipeline Builder**, intended to support the guided construction of data processing pipelines. The main contribution of this work lies in the definition of an abstract model and the implementation of a software architecture capable of ensuring the structural consistency of pipelines, guiding the user during the composition process, and promoting the reuse of elements defined across different application contexts.

The system introduces a formal model based on four fundamental entities:

- *Context*, which represents the logical environment for organizing and reusing elements.
- *ObjectType*, which represents a data type within the system.
- *Algorithm*, which represents a transformation between data types.
- *Pipeline*, which represents an ordered sequence of transformations.

Among these entities, the concept of context plays a central role in the system architecture, since contexts are organized hierarchically and allow the definition of distinct logical environments within which data types, algorithms, and pipelines are managed. This approach provides several advantages:

- Elements defined in a parent context are automatically available in child contexts.
- Elements can be structured hierarchically, separating different application domains.
- Each context can introduce specific elements without interfering with others.
- The system can be extended by introducing new contexts without modifying existing ones.

From a conceptual point of view, the adopted model can be represented as a directed graph, in which nodes correspond to data types (*ObjectType*), edges to transformations (*Algorithm*), and pipelines to valid paths within the graph. Validation mechanisms are also defined on this model, ensuring compatibility between the input and output of each step and preventing the construction of invalid pipelines.

The system also adopts a step-by-step composition approach, which guides the user in choosing valid transformations at each stage of the process, making pipeline construction explicit and controlled.

The system was developed using Python technologies, adopting a modular architecture that separates the different application layers:

- *FastAPI* for implementing the backend and exposing REST APIs.
- *SQLAlchemy* as an Object-Relational Mapper (ORM) for managing persistence in a relational database (SQLite).
- *Streamlit* for developing the user interface.

This architectural choice clearly separates the presentation, application logic, and persistence layers, improving the modularity, maintainability, and extensibility of the system.

The following sections describe the main features of the developed system. First, the report introduces the overall functioning of the system, describing the role of the different components and the operational flow followed by the user when building a pipeline. It then presents the software architecture, analyzes the main implemented modules, and finally discusses realistic usage scenarios to show the behavior of the application in different contexts.

2 Overview and General Functioning of the System

The developed system adopts a layered architecture designed to clearly separate the responsibilities of the different application components and to promote modularity, maintainability, and extensibility. The main functional areas are:

- **API:** represents the access layer of the system and exposes its functionalities through REST endpoints. This module handles incoming requests, validates input data, and delegates operations to the underlying application logic.
- **DB:** contains the persistence models and data access operations. It includes both the ORM entities, which map the database tables, and the functions used to manage data operations, such as CRUD operations (Create, Read, Update, and Delete) and queries.
- **Models:** defines the runtime models used by the application logic. These classes represent the entities directly involved in pipeline construction (*ObjectType*, *Algorithm*, *Pipeline*) independently of the database, separating the logical representation from persistence. The *Context* entity does not have a dedicated runtime model, because it is managed as an organization, filtering, and inheritance mechanism at the DB, API, and user interface levels.
- **Schemas:** collects the validation schemas used for API requests and responses. Based on Pydantic, these schemas ensure the correctness of the data exchanged between client and server.
- **Algorithm catalog:** manages and indexes the available algorithms. It provides optimized data structures for quickly retrieving compatible transformations during pipeline construction.
- **User interface (UI):** developed using Streamlit, it represents the interaction point between the user and the system, providing guided and interactive access to the available functionalities.

The interaction between these components follows a guided operational flow that supports the user in building pipelines.

In a typical usage session, the process begins with the selection of a working context through the user interface. The system then provides access to all elements defined in the current context and in its ancestors, using the inheritance mechanism.

Subsequently, the user can define *ObjectType* entities (data types) and *Algorithm* entities (transformations) through the dedicated functionalities. This information is validated by the backend and stored in the database, making it available for the pipeline construction phase.

The central phase of the system is the guided construction of the pipeline. In this phase, the user selects an initial data type and a target data type, thereby starting a composition session. The backend initializes the pipeline state and builds a catalog of the available algorithms, filtered according to the current context.

At this point, the system enters an iterative step-by-step cycle, which can be described as follows:

1. The current data type is determined, initially corresponding to the starting type.
2. All applicable algorithms, namely those compatible with the current type, are identified.
3. Among them, only the algorithms that do not introduce inconsistencies are selected, for example by avoiding cycles or invalid transformations.
4. The possible choices are presented to the user through the interface.
5. The user selects an algorithm, which is then added to the pipeline.

The process is repeated until the target data type is reached or until a deadlock condition occurs, meaning that no further valid transformations are available to reach the target.

During construction, the system maintains the pipeline state and dynamically updates its information, such as the current type and the total cost. Once completed, the pipeline can be saved in the database.

3 Software Architecture of the System

This chapter describes the software architecture of the **Pipeline Builder** system, focusing on its internal structure and on the main design choices adopted during development. The main modules of the system are analyzed by describing their role, internal organization, and interactions.

3.1 API (access and orchestration layer)

The API module represents the access point to the system and exposes its functionalities through REST endpoints. It plays an orchestration role by handling client requests, validating input data, and delegating operations to the application logic and persistence layer. The module follows a component-based organization, where each file is responsible for managing a specific domain entity. This structure keeps the code clear, modular, and easy to extend.

3.1.1 Central router

The `GestoreAPI.py` file defines the main application router, which aggregates all API submodules and organizes their endpoints using specific prefixes.

```
router = APIRouter()

router.include_router(
    object_type_router,
    prefix="/object-types",
    tags=["Object Types"]
)

router.include_router(
    algorithm_router,
    prefix="/algorithms",
    tags=["Algorithms"]
)

router.include_router(
    pipeline_router,
    prefix="/pipelines",
    tags=["Pipelines"]
)

router.include_router(
    context_router,
    prefix="/contexts",
    tags=["Contexts"]
)
```

3.1.2 API Structure

The `ObjectTypeAPI.py`, `AlgorithmsAPI.py`, and `ContextAPI.py` modules follow a similar structure based on three main elements:

- endpoint definition through `APIRouter`;
- use of the `get_db` dependency for database access;
- preliminary validation of data and context.

In particular, these modules systematically check whether the selected context exists and whether the involved elements are valid. For example, before creating an object or an algorithm, the system verifies that the context exists and that all references are consistent.

```
try:
    context_hierarchy = get_context_hierarchy(db, context_id)
except ValueError:
    raise HTTPException(status_code=404, detail="Context not found")
```

ObjectTypeAPI.py The `ObjectTypeAPI.py` module manages CRUD operations on *ObjectTypes*. In addition to basic operations and duplicate checks, it also manages the algorithm catalog cache. Whenever an *ObjectType* is modified, the cache is invalidated, because the catalog is built from the available *ObjectType* and *Algorithm* entities. Therefore, if a data type changes, the transformations applicable during pipeline construction may also change. Cache invalidation forces the system to rebuild the updated catalog at the next request, preventing the use of outdated information.

AlgorithmsAPI.py The `AlgorithmsAPI.py` module manages the definition of transformations between *ObjectTypes*. Compared with `ObjectTypeAPI.py`, it introduces an additional validation layer: it verifies that the input and output types of each algorithm exist in the current context or in one of its parent contexts. This check ensures the semantic correctness of the defined transformations and prevents the creation of non-applicable algorithms.

In this case as well, each modification invalidates the catalog cache.

ContextAPI.py The `ContextAPI.py` module manages contexts and their hierarchical structure. Its main operations include creating, retrieving, and deleting contexts. Unlike the previous modules, this component also introduces specific checks on parent-child relationships. For example, when creating a new context with a parent, the system verifies that the parent context actually exists:

```
if context.parent_id:
    parent = get_context_by_id(db, context.parent_id)
    if not parent:
        raise HTTPException(status_code=404, detail="Parent context not found")
```

This check ensures the validity of the hierarchical structure.

PipelineAPI.py

The `PipelineAPI.py` module represents the central component of the API layer, as it implements the logic for guided pipeline construction. Unlike the other API modules, which mainly provide CRUD operations, this module includes more complex logic based on:

- management of temporary in-memory sessions (`active_sessions`);
- incremental pipeline construction through a step-by-step approach;
- use of an algorithm catalog filtered according to the selected context;
- dynamic validation mechanisms to prevent invalid configurations.

Pipeline construction is managed through a sequence of endpoints:

- `start-composition`: initializes a new composition session;
- `next-steps`: returns the algorithms applicable to the current pipeline state;
- `select-step`: adds a selected algorithm as a new pipeline step;
- `save-pipeline`: saves the completed pipeline in the database.

The pipeline state is managed on the server side, allowing the system to guide the user throughout the composition process and to ensure the validity of each step.

3.2 Algorithm Catalog

The algorithm catalog is a support component used during pipeline construction. Its purpose is to organize the available transformations and make them efficiently accessible, allowing the system to quickly identify the algorithms compatible with a given data type. In this way, it acts as an intermediate layer between the persistence layer and the application logic.

This component is implemented in the `Catalogo.py` file through the `AlgCatalog` class, which provides a specialized data structure for managing and indexing algorithms.

Internally, the class maintains a list of algorithms and two dictionary-based indexes. These indexes allow algorithms to be retrieved efficiently according to their input or output type, without scanning the entire list.

```
self._algorithms = []
self._input_index = defaultdict(list)
self._output_index = defaultdict(list)
```

In particular:

- `_algorithms` contains the complete set of algorithms included in the catalog;
- `_input_index` associates each input type with the list of applicable algorithms;
- `_output_index` associates each output type with the algorithms that produce it.

Algorithm Indexing

Algorithms are added to the catalog through the `add_algorithm` method, which inserts the algorithm into the main list and updates the corresponding indexes.

```
def add_algorithm(self, algorithm: Algorithm):
    if not isinstance(algorithm, Algorithm):
        raise ValueError("algorithm must be an Algorithm")
    if algorithm in self._algorithms:
        return
    self._algorithms.append(algorithm)
    self._input_index[algorithm.input_type].append(algorithm)
    self._output_index[algorithm.output_type].append(algorithm)
```

This method prevents duplicates and keeps the indexing structures consistent.

Search for Compatible Algorithms

One of the main functions of the catalog is the efficient retrieval of algorithms compatible with a given data type. For example, the `get_by_input` method returns all algorithms that accept a specific type as input:

```
def get_by_input(self, object_type: ObjectType):
    if not isinstance(object_type, ObjectType):
        raise ValueError("object_type must be ObjectType")
    return self._input_index.get(object_type, [])
```

A similar method is provided for retrieval by output type. These operations are used directly by the `PipelineAPI.py` module to determine the applicable transformations at each step of the pipeline construction process.

Dynamic Catalog

The catalog is not static: it is built dynamically from the data stored in the database. The `load_from_db` method converts persistent entities into runtime objects and inserts them into the catalog.

```
def load_from_db(self, db_algorithms, valid_object_types=None):
    valid_names = None
    if valid_object_types:
        valid_names = {obj.name for obj in valid_object_types}
    for alg_db in db_algorithms:
        if valid_names:
            if alg_db.input_type not in valid_names or alg_db.output_type not in valid_names:
                continue
            input_type = ObjectType(name=alg_db.input_type)
            output_type = ObjectType(name=alg_db.output_type)
```

```

        algorithm = Algorithm(
            name=alg_db.name,
            input_type=input_type,
            output_type=output_type,
            cost=alg_db.cost,
            id=getattr(alg_db, "id", None)
        )
        self.add_algorithm(algorithm)

```

To support the context mechanism, the method provides an optional filter based on valid *ObjectTypes*. This filter is represented by the `valid_object_types` parameter, which contains the data types available in the current context and in its parent contexts. During loading, an algorithm is inserted into the catalog only if both its input type and output type belong to this set.

As a result, the catalog contains only the transformations that are actually applicable in the selected context.

3.3 Persistence Layer (DB)

The persistence layer is responsible for managing system data and storing it in a relational database. In this implementation, SQLite is used together with SQLAlchemy, which maps domain entities to relational tables and allows them to be handled as Python objects.

This layer is organized into three main components:

- database configuration and session management;
- definition of ORM models;
- operational modules for data access and manipulation.

3.3.1 Database Configuration

The `ConfigDB.py` file defines the database configuration and access mechanisms. In particular, it initializes the connection engine, the session factory, and the base class for the ORM models.

```

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    echo=False
)
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
Base = declarative_base()

```

An important element is the `get_db` function, which is used as an API dependency to obtain a database session:

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

This approach ensures safe connection management by preventing leaks and closing each session properly.

3.3.2 ORM Models

The `EntitaSQL_db.py` file defines the persistent entities of the system through ORM classes. Each class represents a database table and includes both attributes and relationships.

For example, the hierarchical structure of contexts is modeled through a recursive relationship:

```
PARENT_ID_C = Column(Integer, ForeignKey("TBL_CONTEXTS.ID_C"), nullable=True)
parent = relationship("ContextDB", remote_side=[ID_C])
children = relationship("ContextDB", overlaps="parent")
```

Database-level constraints are also defined to ensure data integrity. For example:

```
# for ObjectTypes
__table_args__ = (
    UniqueConstraint('NAME_OT', 'CONTEXT_ID_OT', name='unique_objecttype_per_context'),
)
# for Algorithms
__table_args__ = (
    UniqueConstraint('NAME_A', 'CONTEXT_ID_A', name='unique_algorithm_per_context'),
    CheckConstraint('COST_A >= 0', name='check_cost_non_negative'),
)
```

These constraints prevent duplicate entries and enforce data validity directly at the database level.

3.3.3 Operational Modules

The `OpDB_*.py` modules implement data access operations, including CRUD operations and queries. They all follow a common structure:

- use of the SQLAlchemy session as a parameter;
- input validation;
- use of ORM queries to access data;
- explicit transaction management through commit and rollback operations.

This approach separates data access logic from application logic, improving the modularity of the system.

OpDB_Context.py The `OpDB_Context.py` module manages contexts and their hierarchy. A central element is the `get_context_hierarchy` function, which returns the chain of contexts from the current one up to the root, thereby implementing the inheritance mechanism between contexts.

OpDB_ObjectType.py and OpDB_Algorithm.py The `OpDB_ObjectType.py` and `OpDB_Algorithm.py` modules implement CRUD operations on *ObjectTypes* and algorithms, introducing specific checks related to the context.

In particular, the algorithm module provides a function that retrieves the algorithms available within a context hierarchy, allowing inheritance to be applied to transformations as well:

```
return db.query(AlgorithmDB).filter(
    AlgorithmDB.CONTEXT_ID_A.in_(context_ids)
).all()
```

OpDB_Pipeline.py The `OpDB_Pipeline.py` module is the most complex persistence module, as it manages both pipeline storage and reconstruction into runtime form.

When a pipeline is saved, the logical pipeline is translated into a relational structure composed of a main table and a step table:

```
for i, step in enumerate(pipeline.steps, start=1):
    db_step = PipelineStepDB(
        PIPELINE_ID=db_pipeline.ID_P,
        PIPELINE_NAME=db_pipeline.PIPELINE_NAME,
        ID_STEP=i,
        ALGORITHM_ID=db_alg.ID_A
    )
```

A reconstruction function is also provided. It converts persistent data into runtime objects, implementing a *DB* $\rightarrow$ *logical model* mapping that keeps the architectural layers separate:

```
for step in steps:
    alg = Algorithm(
        name=alg_db.NAME_A,
        input_type=ObjectType(name=alg_db.INPUT_TYPE_A),
        output_type=ObjectType(name=alg_db.OUTPUT_TYPE_A),
        cost=alg_db.COST_A
    )
    pipeline.add_step(alg)
```

3.4 Runtime Models

Runtime models form the logical layer on which the pipeline construction process is based. Unlike the ORM models described in the previous section, these classes are not tied to persistence. Instead, they provide a logical representation of the domain entities directly involved in pipeline construction: *ObjectType*, *Algorithm*, and *Pipeline*.

This separation decouples the pipeline construction logic from the database structure, improving maintainability and making the system more flexible and extensible.

In particular:

- *ObjectType* (in `ObjectType.py`) defines the space of data types;
- *Algorithm* (in `Algorithm.py`) defines the available transformations;
- *Pipeline* (in `Pipeline.py`) implements the controlled composition mechanism.

These classes are used directly by the APIs, especially by the `PipelineAPI.py` module, to build and validate pipelines during step-by-step composition sessions.

Data Types

The *ObjectType* class represents a data type within the system and conceptually corresponds to a node in the transformation graph. Its implementation is intentionally simple, and its main attributes are immutable:

```
class ObjectType:
    def __init__(self, name: str, description: str = "", id: int = None):
        if not name or not isinstance(name, str):
            raise ValueError("ObjectType name must be a non-empty string")
        self.id = id
        self._name = name.strip()
        self._description = description.strip() if description else ""
```

Validation in the constructor ensures that each data type is defined by a valid name. The name is also normalized using `strip()`, preventing inconsistencies caused by leading or trailing spaces. The main attributes, namely `name` and `description`, are exposed as read-only properties, preventing modifications after object creation.

The `__eq__` and `__hash__` methods are also important, as they make *ObjectType* instances comparable and usable in data structures such as sets and dictionaries:

```
def __eq__(self, other):
    if not isinstance(other, ObjectType):
        return False
    return self.name == other.name

def __hash__(self):
    return hash(self.name)
```

Transformations

The `Algorithm` class represents a transformation between two *ObjectTypes*, corresponding to an edge in the pipeline graph. Each algorithm is defined by:

- an identifying name;
- an input type;
- an output type;
- an associated cost.

The constructor performs strict parameter checks to ensure that each transformation is semantically consistent and that the cost is always valid:

```
class Algorithm:
    def __init__(
        self,
        name: str,
        input_type: ObjectType,
        output_type: ObjectType,
        cost: float = 1.0,
        id: int = None
    ):
        if not name or not isinstance(name, str):
            raise ValueError("Algorithm name must be a non-empty string")
        if not isinstance(input_type, ObjectType):
            raise ValueError("input_type must be ObjectType")
        if not isinstance(output_type, ObjectType):
            raise ValueError("output_type must be ObjectType")
        if not isinstance(cost, (int, float)) or cost < 0:
            raise ValueError("cost must be a non-negative number")
```

In this case as well, the attributes are exposed through read-only properties, keeping the object immutable after creation. This model is central to the pipeline construction logic, as it defines the possible transitions between data types and allows compatibility between steps to be verified.

Step Composition and Validation

The `Pipeline` class represents an ordered sequence of transformations, corresponding to a valid path in the graph defined by *ObjectTypes* and algorithms. Each pipeline consists of a list of steps, represented by the `PipelineStep` class, which encapsulates a single transformation and ensures consistency between the input type and the selected algorithm:

```
class PipelineStep:
    def __init__(self, input_type: ObjectType, algorithm: Algorithm):
        if not isinstance(input_type, ObjectType):
            raise ValueError("input_type must be ObjectType")
        if not isinstance(algorithm, Algorithm):
            raise ValueError("algorithm must be Algorithm")
        if algorithm.input_type != input_type:
            raise ValueError("Algorithm is not compatible with input")
        self._input_type = input_type
        self._algorithm = algorithm
        self._output_type = algorithm.output_type
```

The `Pipeline` class manages the full sequence of steps through the `add_step` method, which extends the pipeline by adding a new algorithm. Internally, the system determines the current type, corresponding to the output of the last step, and automatically verifies compatibility. During construction, the pipeline therefore maintains a consistent state and dynamically updates the current type.

In addition to these functionalities, the class also supports:

- calculation of the total pipeline cost;
- a readable representation of the transformation sequence.

3.5 Validation Schemas

The `Schemas` layer is responsible for validating and structuring the data exchanged between client and server through the REST APIs. It acts as an interface between the client and the system, ensuring that all interactions follow well-defined constraints. In particular:

- it ensures the correctness of input data, protecting the system from invalid inputs;
- it explicitly defines the format of requests and responses;
- it enables automatic integration with FastAPI's Swagger documentation;
- it decouples the API layer from the internal runtime and ORM models.

This layer is implemented using the Pydantic library, where all schemas are defined as classes extending `BaseModel`. The fields are typed and enriched with constraints through `Field`, which allows properties such as minimum length, allowed values, and descriptions to be specified. Furthermore, each schema uses `ConfigDict` to configure additional behaviors, such as:

- `json_schema_extra`: provides examples in the documentation;
- `from_attributes=True`: enables automatic conversion from ORM objects.

ObjectType Schema

The `ObjectType_schema.py` file defines the schemas related to data types. The creation schema, `ObjectTypeCreate`, specifies the fields required to define a new type:

```
class ObjectTypeCreate(BaseModel):
    name: str = Field(..., min_length=1, description="ObjectType name")
    description: str = Field(default="", description="Description (optional)")
```

For responses, the `ObjectTypeResponse` schema defines the structure of the data returned by the APIs: id, name, description, and associated context, including both ID and name. A configuration is also provided to automatically convert ORM objects into serializable output:

```
model_config = ConfigDict(
    from_attributes=True
)
```

This option is particularly useful because it allows objects from the persistence layer, such as SQLAlchemy instances, to be returned without manually converting them into dictionaries. The serialization process is delegated to Pydantic, reducing redundant code and limiting the possibility of errors.

Algorithm Schema

The `Algorithm_schema.py` file defines the schemas for managing algorithms. The creation schema, `AlgorithmCreate`, includes the fundamental fields of a transformation:

```
class AlgorithmCreate(BaseModel):
    name: str = Field(..., min_length=1, description="Algorithm name")
    input_type: str = Field(..., min_length=1, description="Input type")
    output_type: str = Field(..., min_length=1, description="Output type")
    cost: float = Field(..., ge=0, description="Algorithm cost")
```

The response schema, `AlgorithmResponse`, returns the relevant information, including context-related data: id, name, input type, output type, cost, context ID, and context name. Here too, the `from_attributes=True` configuration is used for automatic conversion from ORM objects.

Pipeline Schema

The `Pipeline_schema.py` file is the most structured schema module, as it manages different types of requests and responses related to pipeline construction and management.

The `PipelineBuildRequest` schema defines the parameters required to start a new composition session, with two mandatory fields representing the initial and final types of the pipeline.

```
class PipelineBuildRequest(BaseModel):
    start_type: str = Field(..., min_length=1, description="Start type")
    end_type: str = Field(..., min_length=1, description="End type")
```

The `PipelineStepResponse` schema represents a single pipeline step, allowing the pipeline to be serialized in a structured form that is easily interpretable by the client:

```
class PipelineStepResponse(BaseModel):
    input_type: str
    algorithm: str
    output_type: str
```

The `PipelineResponse` schema represents the complete response returned by the APIs. It includes all relevant information, such as pipeline id and name, initial and final type, total cost, sequence of steps, and associated context. In this case as well, automatic conversion from ORM objects is enabled through `from_attributes=True`.

Additional schemas are also defined to support guided pipeline construction and the saving of completed pipelines in the database.

3.6 User Interface

The user interface layer represents the direct interaction point between the user and the system, forming the presentation layer of the architecture. Its purpose is to connect the user to the application logic implemented in the backend, providing a simple, interactive, and guided environment for using the available functionalities. The interface was developed using Streamlit and is implemented in the `InterfacciaUtente.py` file. It is based on a single-page architecture, where the different functionalities are managed dynamically through the application state (`st.session_state`), which is discussed in more detail in the following section. This approach preserves the user context across different operations and supports complex interactive flows, such as step-by-step pipeline construction.

From an architectural point of view, the UI is designed as a component that is fully decoupled from business logic and data persistence. It does not contain computational logic related to pipeline construction; instead, it orchestrates user interactions and delegates all operations to the backend through HTTP calls to the REST endpoints.

In particular, the interface is responsible for:

- invoking the REST endpoints exposed by the backend through HTTP requests;
- presenting data in a structured and readable form;
- managing user interaction, including inputs, selections, and navigation;
- maintaining the session state on the client side.

Communication with the backend is handled through the `requests` library, which is used to call the FastAPI endpoints (`API_URL = "http://127.0.0.1:8000"`). Data are exchanged using the HTTP protocol and JSON-formatted structures.

The interface is organized into different functional sections, accessible through a navigation sidebar, which reflect the main functionalities of the system:

- context management;
- management of *ObjectTypes* and algorithms;
- guided pipeline construction;
- visualization of saved pipelines.

Finally, an initial onboarding phase introduces the user to the system and guides the first steps.

3.6.1 General Structure of the Interface

Streamlit allows interactive web applications to be developed directly in Python. In the developed system, a reactive execution model was adopted: at each user interaction, the script is rerun, while the application state is preserved through the session mechanism.

The initial configuration of the application defines the page title and interface layout, while additional graphical customizations are applied by injecting CSS code through `st.markdown`.

The application state is managed through `st.session_state`, which maintains persistent information across different executions of the script. In particular, it manages:

- the application startup state (`app_started`);
- the active context (`active_context_id`);
- the current view (`current_view`);
- the state of the pipeline under construction, such as `session_id`.

State initialization is performed through explicit checks, allowing navigation logic to be implemented without using a traditional routing system:

```
if "app_started" not in st.session_state:
    st.session_state.app_started = False
```

At application startup, an introductory screen is shown to present the system and guide the user through the first steps. The transition to the operational phase is handled by a button that updates the application state and reruns the interface:

```
if st.button("Next"):
    st.session_state.app_started = True
    st.session_state.current_view = "context"
    st.rerun()
```

Navigation between the different functionalities is managed through a sidebar, which represents the main access point to the application sections. It shows the active context, provides the context selection button, includes the navigation menu, and gives access to the tutorial. The current view is selected by updating the application state:

```
if st.sidebar.button("Change Context"):
    st.session_state.current_view = "context"
    st.rerun()
```

The main menu is implemented through interactive components that allow the active section of the interface to be selected dynamically. The interface does not access the database directly; instead, it delegates all operations to the backend and only manages presentation and user interaction.

Although the application is contained in a single file, the different functionalities are organized into dedicated functions, which are invoked according to the current state. This keeps the structure modular and readable even within a single-page architecture.

3.6.2 Context Management

Context management represents the main entry point of the application, since every subsequent operation is performed within an active context. The user interface therefore provides tools to select, create, and delete contexts, as well as to visualize their hierarchical structure.

At application startup, the user is guided toward context selection through a dedicated screen, where the available contexts retrieved from the backend through an HTTP call are shown:

```
res = requests.get(f"{API_URL}/contexts")
contexts = res.json() if res.status_code == 200 else []
```

The active context is then stored in the application state through the `active_context_id` variable, which is used in all subsequent requests.

The context management interface is organized through a tab system that divides the main functionalities into three sections:

- visualization of the available contexts;
- creation of a new context;
- deletion of an existing context.

Contexts are displayed through a structured list showing their names and identifiers, allowing the user to select the desired context. The selection updates the application state and redirects the user to the main application view.

A new context is created through a form where the user specifies its name and, optionally, a parent context. This reflects the hierarchical model defined in the backend and allows tree structures to be built:

```
res = requests.post(
    f"{API_URL}/contexts",
    json={"name": new_name, "parent_id": parent_id}
)
```

Context deletion is managed by selecting an existing context from a list and sending an HTTP DELETE request, limited to the available contexts.

A distinctive feature of this section is the visualization of the context hierarchy, represented in textual form as an ASCII tree. This representation is built dynamically from the data received from the backend, organizing contexts according to their parent-child relationships and producing a readable hierarchical view.

Finally, the active context is always shown in the application sidebar, ensuring that the user is aware of the current working environment. The context can be changed at any time, updating the state and rerunning the interface.

3.6.3 ObjectType and Algorithm Management

The management of *ObjectTypes* and algorithms allows the user to define data types and the transformations between them. Both entities are closely linked to the active context, whose identifier is included in every request to the backend.

From the interface point of view, the two functionalities share a common structure, organized through a tab system that divides the main operations into three sections:

- visualization of the available elements;
- creation of a new element;
- deletion of an existing element.

Visualization is performed through HTTP requests to the backend, filtered by the active context:

```
requests.get(f"{API_URL}/object-types/",
             params={"context_id": context_id}
)
requests.get(f"{API_URL}/algorithms/",
             params={"context_id": context_id}
)
```

The returned data are presented in tabular form, showing the main information for each element. For *ObjectTypes*, the table displays the name and description, while for algorithms it also shows the input type, output type, and cost. Algorithm management introduces an additional constraint compared with *ObjectTypes*: the user must specify an input type and an output type among the *ObjectTypes* available in the active context, ensuring the consistency of the defined transformations.

Creation is performed through dedicated forms, which collect the required information and send an HTTP POST request:

```
requests.post(f"{API_URL}/object-types/",
              params={"context_id": context_id},
              json={...}
            )
requests.post(f"{API_URL}/algorithms/",
              params={"context_id": context_id},
              json={...}
            )
```

Element deletion is limited to objects belonging to the active context, in order to preserve the consistency of data inherited from higher-level contexts. The operation is performed through HTTP DELETE requests:

```
requests.delete(f"{API_URL}/object-types/{name}",
               params={"context_id": context_id}
            )
requests.delete(f"{API_URL}/algorithms/{name}",
               params={"context_id": context_id}
            )
```

3.6.4 Pipeline Builder

The *Pipeline Builder* section is the central component of the user interface and, more generally, of the entire application. It enables the guided construction of pipelines through a step-by-step process. This process directly reflects the logic implemented in the backend, allowing the user to compose a sequence of valid transformations from an initial type to a desired final type.

Pipeline construction is performed within the active context and is based on a composition session, identified by a `session_id`, which is maintained in the application state. This allows the system to track the evolution of the pipeline across user interactions.

The process begins with the selection of the desired input type and output type. These values are chosen among the *ObjectTypes* available in the active context and sent to the backend through an initialization request:

```
response = requests.post(
    f"{API_URL}/pipelines/start-composition",
    params={
        "start_type": start_type,
        "end_type": end_type,
        "context_id": context_id
    },
    timeout=5
)
```

If the request is successful, the backend returns a session identifier, which is saved in `st.session_state`. From this moment on, the pipeline construction process can continue. At each iteration, the interface requests from the backend the list of algorithms applicable to the current type:

```
response = requests.get(
    f"{API_URL}/pipelines/next-steps/{st.session_state.session_id}",
    timeout=5
)
```

The available algorithms are presented to the user, who selects the desired one. The selection is then sent to the backend to update the pipeline state:

```
res = requests.post(
    f"{API_URL}/pipelines/select-step/{st.session_state.session_id}",
    json={"algorithm_name": alg["name"]},
    timeout=5
)
```

The process continues until the desired output type is reached or until no further valid transformations are available.

During construction, the interface shows the current state of the pipeline, including the sequence of selected steps and the accumulated total cost. This allows the user to monitor the pipeline evolution and make informed decisions when choosing algorithms.

Once the pipeline has been completed, the user can save it by assigning it a name. This operation is performed through an HTTP request:

```
res = requests.post(
    f"{API_URL}/pipelines/save-pipeline/{st.session_state.session_id}",
    json={"name": st.session_state.pipeline_name},
    timeout=5
)
```

After the saving process, the pipeline is persisted in the database and becomes available for consultation in the dedicated sections.

3.6.5 Management of Saved Pipelines

The section dedicated to saved pipelines allows the user to consult and manage the pipelines previously created and stored in the system. Here too, all operations are filtered according to the active context.

The list of pipelines is retrieved from the backend through an HTTP request:

```
response = requests.get(
    f"{API_URL}/pipelines/",
    params={"context_id": context_id}
)
```

The returned data are presented in a structured form, showing the main information for each pipeline, including its name, initial type, final type, and total cost.

For each pipeline, the interface also displays the sequence of steps returned by the backend. The steps are shown in order through the `render_pipeline_table` function, indicating the input type, applied algorithm, and output type for each step:

```
if p["steps"]:
    html += render_pipeline_table(p["steps"])
else:
    html += "<div style='color:#999;'>No steps</div>"
```

The interface also allows the user to delete a saved pipeline. The user selects the desired pipeline, and the operation is performed through an HTTP DELETE request limited to the active context.

3.7 System Entry Point

The entry point of the system is defined in the `main.py` file, which represents the starting point of the FastAPI-based backend application. This module initializes the application, configures the database, and registers all endpoints exposed by the system.

The application is created by instantiating the `FastAPI` class, where descriptive information is defined for documentation purposes and for the automatic generation of the OpenAPI documentation:

```
app = FastAPI(  
    title="Pipeline Builder API",  
    description="API for building and managing data processing pipelines",  
    version="1.0.0"  
)
```

This configuration automatically provides interactive documentation through Swagger UI, facilitating API testing and exploration.

A fundamental aspect of initialization concerns database management. Through the `@app.on_event("startup")` decorator, a function is defined and automatically executed when the server starts. Its purpose is to ensure that the required tables are available:

```
@app.on_event("startup")  
def on_startup():  
    Base.metadata.create_all(bind=engine)
```

In this way, the system keeps the database structure consistent with the defined ORM models and avoids the need for manual initialization operations at runtime.

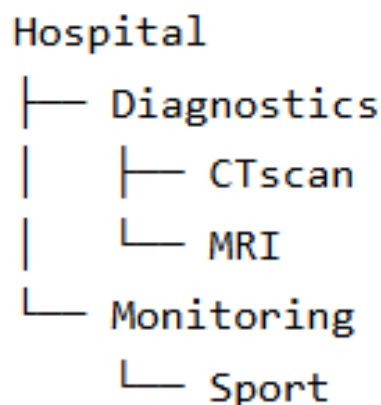
Finally, the module registers the main application router, defined in the `GestoreAPI.py` file, which aggregates all API submodules related to the different domain entities: `app.include_router(router)`.

This operation makes all endpoints available through a single access point while preserving a clear modular separation within the code.

4 Usage Examples

This chapter presents a set of realistic usage scenarios to demonstrate how the developed system works in practice. The goal is to illustrate the use of the interface and show how the proposed model can represent real data transformation processes in an explicit and structured way.

The system is assumed to be initialized with a set of hierarchically organized contexts representing different application domains. In particular, there is a root context named **Hospital**, from which several specialized subcontexts derive, including **Diagnostics**, **Monitoring**, **CTscan**, **MRI**, and **Sport**.



Regarding data types (*ObjectTypes*), the defined types are:

ID_OT	NAME_OT	CONTEXT_ID_OT	CONTEXT_NAME_OT	DESCRIPTION_OT
1	RawData	1	Hospital	
2	CleanData	1	Hospital	
3	FeatureData	1	Hospital	
4	Report	1	Hospital	
5	MedicalImage	2	Diagnostics	
6	ProcessedImage	2	Diagnostics	
7	Diagnosis	2	Diagnostics	
8	CTscan_Image	4	CTscan	
9	CTscan_Processed	4	CTscan	
10	CTscan_Diagnosis	4	CTscan	
11	RM_Image	5	MRI	
12	RM_Processed	5	MRI	
13	RM_Diagnosis	5	MRI	
14	SensorData	3	Monitoring	
15	TimeSeries	3	Monitoring	
16	Alert	3	Monitoring	
17	GPSData	6	Sport	
18	ActivityPattern	6	Sport	
19	PerformanceReport	6	Sport	

The defined transformations are:

ID_A	NAME_A	CONTEXT_ID_A	CONTEXT_NAME_A	INPUT_TYPE_A	OUTPUT_TYPE_A	COST_A
1	clean_data	1	Hospital	RawData	CleanData	1.0
2	extract_features	1	Hospital	CleanData	FeatureData	1.0
3	generate_report	1	Hospital	FeatureData	Report	1.0
4	convert_to_image	2	Diagnostics	RawData	MedicalImage	1.0
5	enhance_image	2	Diagnostics	MedicalImage	ProcessedImage	1.0
6	diagnose	2	Diagnostics	ProcessedImage	Diagnosis	2.0
7	tac_reconstruction	4	CTscan	MedicalImage	CTscan_Image	2.0
8	tac_filtering	4	CTscan	CTscan_Image	CTscan_Processed	1.0
9	tac_analysis	4	CTscan	CTscan_Processed	CTscan_Diagnosis	2.0
10	rm_reconstruction	5	MRI	MedicalImage	RM_Image	2.0
11	rm_denoising	5	MRI	RM_Image	RM_Processed	1.0
12	rm_analysis	5	MRI	RM_Processed	RM_Diagnosis	2.0
13	normalize_sensor	3	Monitoring	RawData	SensorData	1.0
14	build_timeseries	3	Monitoring	SensorData	TimeSeries	1.0
15	detect_anomaly	3	Monitoring	TimeSeries	Alert	2.0
16	gps_processing	6	Sport	SensorData	GPSData	1.0
17	pattern_detection	6	Sport	GPSData	ActivityPattern	1.0
18	performance_eval	6	Sport	ActivityPattern	PerformanceReport	1.0

The system already includes some pipelines, which represent complete examples of transformation processes:

ID_P	PIPELINE_NAME	CONTEXT_ID_P	CONTEXT_NAME_P	START_TYPE_ID_P	END_TYPE_ID_P	TOTAL_COST_P
1	Generic_pipeline	1	Hospital	1	4	3.0
2	pipeline_Diagnostics	2	Diagnostics	1	7	4.0
3	pipeline_monitoraggio	3	Monitoring	1	16	4.0
4	pipeline_CTscan	4	CTscan	1	10	6.0
5	pipeline_RM	5	MRI	1	13	6.0
6	pipeline_attivita_fisica	6	Sport	1	19	4.0

When the application starts, an introductory page is displayed, including a short tutorial on how to use the system:

Welcome to Pipeline Builder

Pipeline Builder is an application for guided design of data processing pipelines. It allows you to define data types (*Object Types*) and transformation algorithms, and combine them to build consistent pipelines step by step.

How it works

At startup, you will be asked to choose an operating context, and after that, you will have access to the main menu:

- Object Types: View/Define/Delete data types
- Algorithms: View/Define/Delete transformations between data types
- Saved Pipelines: View and reuse previously created pipelines
- Pipeline Builder: Build pipelines step by step, choosing the initial and the final type. The system will suggest only valid transformations

Context inheritance

Elements (*Object Types, Algorithms, Pipelines*) defined in a parent context are automatically available in child contexts. A pipeline created in a higher-level context can therefore be reused in derived contexts.

 Click **Next** to select a context and start building your pipeline.

Next

To proceed, the user clicks the *Next* button.

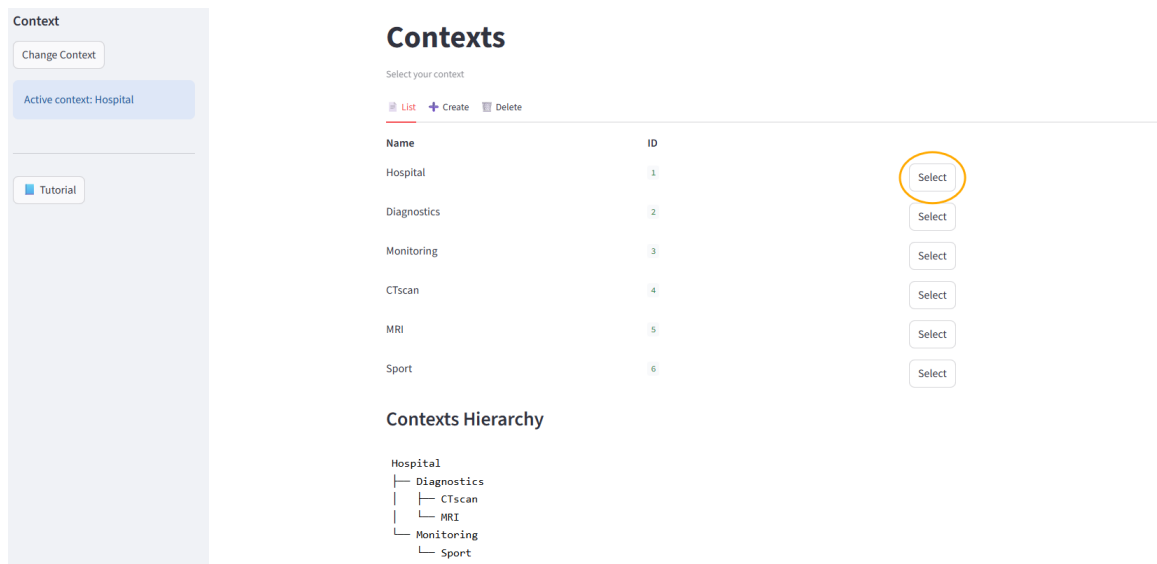
4.1 Scenario 1: Generic Pipeline

Consider a hospital environment in which a physician needs to analyze data related to a newly admitted patient.

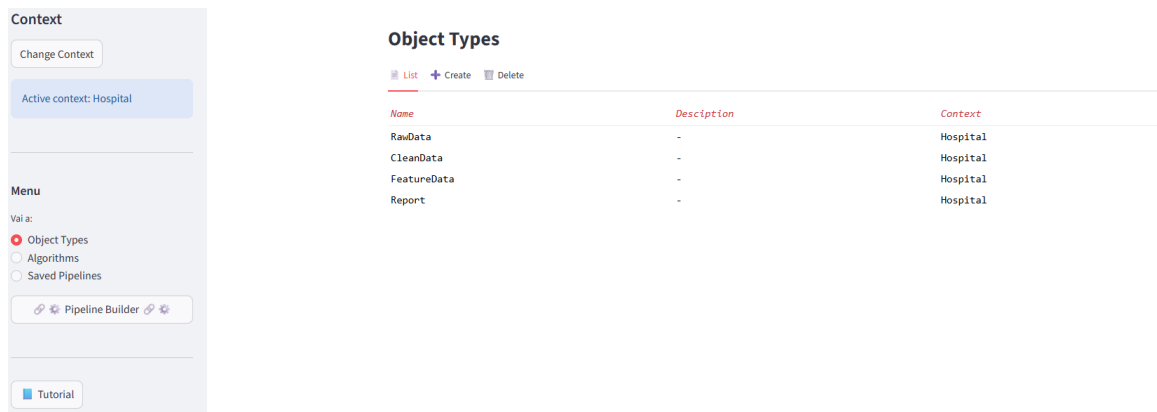
The patient has undergone a series of preliminary examinations, whose results are available in heterogeneous formats (*RawData*), including clinical measurements, laboratory values, and unstructured annotations contained in the medical documentation.

Before drawing any conclusions, the physician needs to reorganize and integrate this information into a more interpretable form, in order to obtain a structured summary useful for an initial clinical assessment.

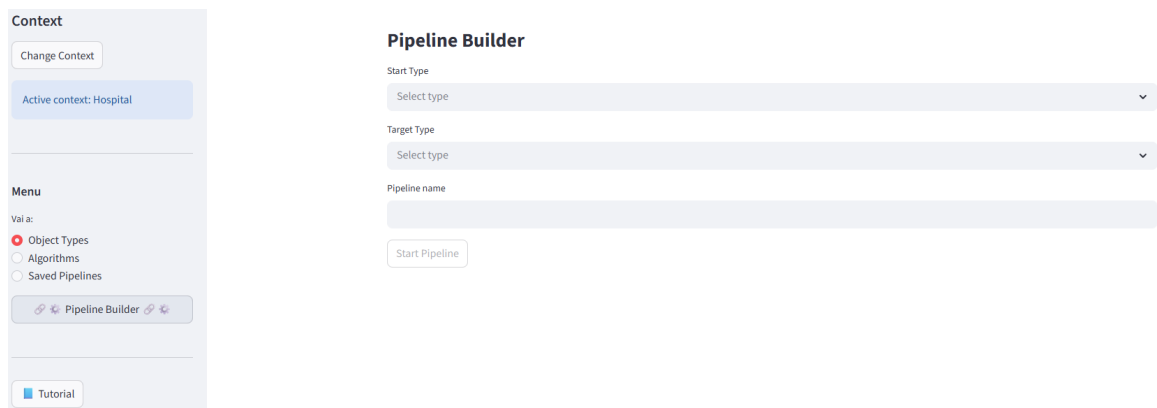
The physician accesses the application and selects the **Hospital** context, which represents the general hospital domain and provides the basic tools for manipulating clinical data.



Once selected, the active context is shown in the interface, in the box in the top-left corner, clearly indicating the operating environment in which all subsequent operations will be performed:



The physician then clicks the *Pipeline Builder* button to build a pipeline that transforms the patient's heterogeneous data (*RawData*) into a synthetic report (*Report*), which can later support clinical decisions or be shared with other specialists.



Starting from the initial type *RawData*, the system automatically proposes the available transformations. During pipeline construction, the interface provides visual support by showing the current data type in the box on the right. Initially, this corresponds to the starting type (*RawData*), but it is dynamically updated after each selected step.

Pipeline Builder

Start Type

RawData

Target Type

Report

Pipeline name

 Restart pipeline

Current Pipeline

Pipeline Empty

CURRENT TYPE

RawData

Possible Algorithms:

clean_data → Cost: 1.0

Select

This allows the physician to keep the state of the ongoing transformation clear and to understand which operations are applicable in the current situation.

The physician then selects, step by step, the algorithms needed to reach the target:

- *clean_data*: removes inconsistencies, missing values, and noise from the clinical data.
- *extract_features*: transforms the cleaned data into a set of relevant features.
- *generate_report*: summarizes the extracted information into a structured report.

During algorithm selection, the system also updates the overall cost of the pipeline in real time and shows it in the interface. This allows the physician to evaluate not only the correctness of the transformations, but also their overall impact in terms of cost.

The constructed pipeline is therefore:

RawData → CleanData → FeatureData → Report

Pipeline Builder

Start Type

RawData

Target Type

Report

Pipeline name

 Restart pipeline

Current Pipeline

1 RawData → clean_data → CleanData
2 CleanData → extract_features → FeatureData
3 FeatureData → generate_report → Report

TOTAL COST

3.0

Pipeline completed

 Save pipeline

 Back

After verifying the result, the physician analyzes the entire process, evaluating its consistency with the considered clinical case and its sustainability in terms of overall cost.

Considering the procedure adequate, the physician decides to save it with the name *PatientDataAnalysis*, so that it can be reused in the future for patients with similar characteristics.

Saved Pipelines

Delete Pipeline

✓ Pipeline "PatientDataAnalysis" (from "RawData" to "Report") saved!

Name	Start Type	Context Name
Generic_pipeline	RawData	Hospital

Cost	Target Type	Context ID
3.0	Report	1

Steps

```
graph LR
  1[1 RawData] --> clean_data[clean_data]
  clean_data --> CleanData[CleanData]
  2[2 CleanData] --> extract_features[extract_features]
  extract_features --> FeatureData[FeatureData]
  3[3 FeatureData] --> generate_report[generate_report]
  generate_report --> Report[Report]
```

Name	Start Type	Context Name
PatientDataAnalysis	RawData	Hospital

Cost	Target Type	Context ID
3.0	Report	1

Steps

```
graph LR
  1[1 RawData] --> clean_data[clean_data]
  clean_data --> CleanData[CleanData]
  2[2 CleanData] --> extract_features[extract_features]
  extract_features --> FeatureData[FeatureData]
  3[3 FeatureData] --> generate_report[generate_report]
  generate_report --> Report[Report]
```

4.2 Scenario 2: Performance Analysis

Consider a sports context in which a coach wants to analyze an athlete's performance during a training session. The collected data come from wearable devices and sensors used during physical activity, but they are initially available in raw form, represented by the *RawData* ObjectType.

The coach's goal is to transform these data into a structured performance report, represented by the *PerformanceReport* ObjectType, which is useful for evaluating the progress of the training session and identifying relevant patterns.

The coach therefore selects the **Sport** context. This context not only contains elements defined directly within it, but can also use the *ObjectTypes* and algorithms inherited from the ancestor contexts **Monitoring** and **Hospital**.

Context

Change Context

Active context: Hospital

Tutorial

Contexts

Select your context

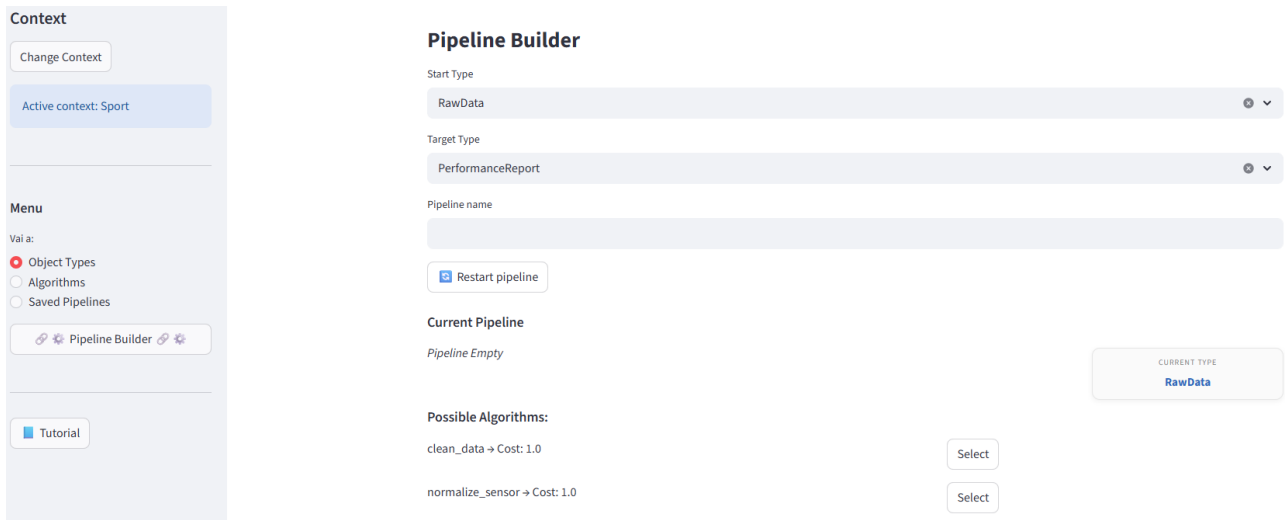
List Create Delete

Name	ID	
Hospital	1	Select
Diagnostics	2	Select
Monitoring	3	Select
CTscan	4	Select
MRI	5	Select
Sport	6	Select

Contexts Hierarchy

```
graph TD
  Hospital --> Diagnostics
  Hospital --> CTscan
  Hospital --> MRI
  Hospital --> Monitoring
  Monitoring --> Sport
```

Once the context has been selected, the coach accesses the *Pipeline Builder* section to build a pipeline that transforms *RawData* into *PerformanceReport*.



Starting from the initial type *RawData*, the system automatically proposes only the transformations compatible with the current type. The inheritance mechanism allows the system to use algorithms and *ObjectTypes* defined in different contexts. In particular, the pipeline combines elements from the **Hospital**, **Monitoring**, and **Sport** contexts:

- *RawData*: *ObjectType* defined in the **Hospital** context;
- *SensorData*: *ObjectType* defined in the **Monitoring** context;
- *GPSSData*, *ActivityPattern*, and *PerformanceReport*: *ObjectTypes* defined in the **Sport** context.

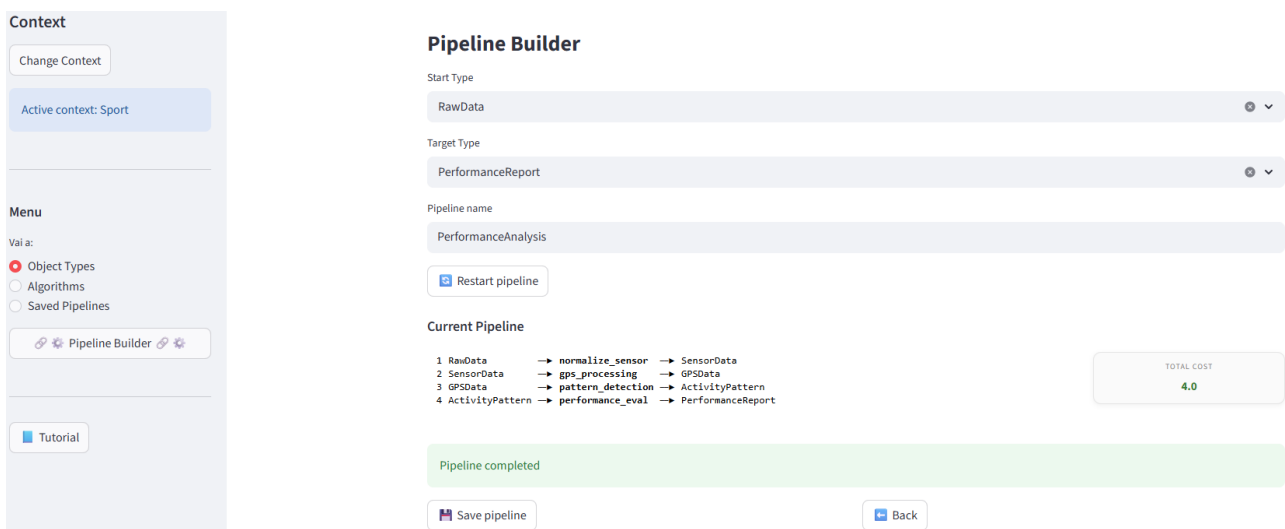
The coach selects the following algorithms step by step:

- *normalize_sensor*: algorithm defined in the **Monitoring** context, which transforms *RawData* into *SensorData*;
- *gps_processing*: algorithm defined in the **Sport** context, which transforms *SensorData* into *GPSSData*;
- *pattern_detection*: algorithm defined in the **Sport** context, which transforms *GPSSData* into *ActivityPattern*;
- *performance_eval*: algorithm defined in the **Sport** context, which transforms *ActivityPattern* into *PerformanceReport*.

During pipeline construction, the system shows the current data type and dynamically updates the available transformations. The coach then selects the algorithms proposed by the system, following the valid path until the final type is reached.

The resulting pipeline is the following:

$RawData \rightarrow SensorData \rightarrow GPSSData \rightarrow ActivityPattern \rightarrow PerformanceReport$



This scenario illustrates the inheritance mechanism between contexts: in the **Sport** context, it is possible to build a pipeline that starts from a general **ObjectType** defined in **Hospital**, uses a transformation and an **ObjectType** from the **Monitoring** context, and ends with transformations specific to the sports domain. In this way, the specialized **Sport** context can reuse more general elements without redefining them.

4.3 Scenario 3: Reuse of Saved Pipelines

Consider a radiology department in which a physician needs to quickly analyze data from a magnetic resonance imaging examination. The patient has just undergone an MRI scan, and the system initially contains the raw data produced by the acquisition, represented by the *RawData* **ObjectType**. The goal is to obtain a structured assessment specific to the examination, represented by the *RM_Diagnosis* **ObjectType**, which can be used as preliminary support for clinical decision-making.

In this scenario, instead of building a new pipeline step by step, the physician decides to use a process already saved in the system. The physician therefore accesses the application and selects the *MRI* context, dedicated to the management of transformations related to magnetic resonance imaging examinations.

Once the working context has been set, the user accesses the *Saved Pipelines* section, where the pipelines previously built and stored in the database are listed.

Among the available pipelines, the user identifies *pipeline_RM*, which transforms raw data into a structured MRI diagnosis. The pipeline is consistent with the considered case, since it has *RawData* as its initial type and *RM_Diagnosis* as its final type.

The screenshot shows a software interface with a sidebar on the left and a main content area on the right. The sidebar has a 'Context' section with a 'Change Context' button and 'Active context: MRI'. Below it is a 'Menu' section with 'Val a:' and three radio buttons: 'Object Types', 'Algorithms', and 'Saved Pipelines' (which is selected). There is also a 'Pipeline Builder' button and a 'Tutorial' button. The main content area displays a list of pipelines. The selected pipeline, 'pipeline_RM', is highlighted in grey. It has a 'Name' of 'pipeline_RM', a 'Start Type' of 'RawData', a 'Target Type' of 'RM_Diagnosis', a 'Context Name' of 'MRI', and a 'Context ID' of '5'. Its 'Cost' is '6.0'. The 'Steps' are: 1. RawData → convert_to_image → MedicalImage; 2. MedicalImage → rm_reconstruction → RM_Image; 3. RM_Image → rm_denoising → RM_Processed; 4. RM_Processed → rm_analysis → RM_Diagnosis. Other pipelines shown include 'pipeline_Diagnostics' (Cost 4.0, Context Name Diagnostics, Context ID 2) and 'PatientDataAnalysis' (Cost 3.0, Context Name Hospital, Context ID 1).

The system shows the main information associated with the selected pipeline, including the pipeline name, initial type, final type, associated context, overall cost, and ordered sequence of steps.

By analyzing the pipeline, the physician can verify the entire transformation flow applied to the data:

- *convert_to_image*: converts the raw data acquired during the examination into a preliminary image representation, making the scan information usable for subsequent diagnostic processing.
- *rm_reconstruction*: reconstructs the magnetic resonance image from the initial representation, producing a structure more suitable for radiological analysis and consistent with the MRI domain.
- *rm_denoising*: applies noise reduction operations to the reconstructed image, improving its visual quality and highlighting the relevant anatomical structures more clearly.
- *rm_analysis*: analyzes the processed MRI image to extract clinically significant information and produce a preliminary structured diagnosis, which can support the physician in evaluating the examination.

This scenario shows how the system can support a realistic healthcare use case by allowing already defined processes to be retrieved and applied to new examinations without manually reconstructing the entire sequence of transformations. In this way, the reuse of saved pipelines promotes speed, consistency, and greater control over the analysis process.

5 Conclusions

The project led to the development of a software system for the guided construction of data processing pipelines, with the aim of supporting the user in defining coherent, modular, and reusable sequences of transformations.

The system addresses the complexity of data preparation by proposing an explicit model based on data types, algorithms, and pipelines. Each transformation is represented in a structured way by specifying its input type, output type, and associated cost, making the composition process more controllable and interpretable.

A central aspect of the system is step-by-step validation: the user does not build an arbitrary sequence freely, but is guided by the system in selecting only the transformations compatible with the current state of the pipeline. In this way, inconsistencies between steps are reduced, and the path followed by the data, from the initial form to the final result, becomes clearer.

The system also introduces the concept of context, which allows *ObjectTypes*, algorithms, and pipelines to be organized into distinct and hierarchical application domains. This mechanism enables the reuse of elements defined in more general contexts within more specific ones.

Future Developments

A first possible development concerns the integration of executable algorithms within the application. In the current version, algorithms are represented only through descriptive information, namely name, input type, output type, and cost. In the future, each algorithm could be associated with functions, scripts, or software modules that can actually be executed, transforming the pipeline from a logical structure into an operational process.

Starting from this extension, it would be possible to introduce a pipeline execution engine. This component would traverse the steps defined by the user and apply the selected algorithms in sequence, passing the output of one step as the input of the next one. In this way, the system would not be limited to pipeline design, but could also support pipeline execution.

A further development concerns the management of real input and output data, also in communication with external systems. For example, other services could send data to the system through dedicated APIs, request the execution of a saved pipeline, and receive the produced result.

The graphical representation of the context hierarchy could also be improved. Currently, the visualization is functional, but it could be made more intuitive and interactive, for example through a more advanced tree representation.

Finally, another possible improvement concerns the refactoring of the user interface code. The *InterfacciaUtente.py* file is organized into dedicated functions and is sufficiently readable, but it contains many responsibilities within a single module. It would therefore be appropriate to divide it into multiple files to make future maintenance and extensions easier.

References

- [1] G. Alfano, I. Bartolini, D. Calvanese, P. Ciaccia, S. Greco, D. Lanti, E. Lenzi, D. Martinenghi, C. Molinaro, M. Patella, L. Tanca, R. Torlone, I. Trubitsyna, *S-PIC4CHU: Semantics-based Provenance, Integrity, and Curation for Consistent, High-quality, and Unbiased Data Science*, SEBD 2025: 33rd Symposium on Advanced Database Systems, 2025.
- [2] G. Alfano, I. Bartolini, D. Calvanese, P. Ciaccia, S. Greco, D. Lanti, P. L. Lazzaro, E. Lenzi, D. Martinenghi, C. Molinaro, M. Patella, L. Tanca, R. Torlone, I. Trubitsyna, *S-PIC4CHU: Semantics-Enriched Techniques for Data Preparation in Data Science*, ITADATA 2025: The 4th Italian Conference on Big Data and Data Science, 2025.
- [3] D. Ciuffreda, M. Ciammaichella, *Multimedia Data Analysis: a Pipeline for the New European Bauhaus KPIs*, Master's Degree Thesis in Computer Science and Engineering, Politecnico di Milano, A.Y. 2025–2026.